Bantay

How the app works

A guide to Bantay's code: how it is organised, how data flows through it, and the rules that keep it working. Written for the developer who built it.

React Native · Expo SDK 57 · Expo Router · TypeScript · Offline-first · AES-256-GCM

Repository: pork-arie/Bantay-app · App version 1.0.0 · October 2026

Contents

1 The big picture

2 Folder structure

3 Navigation: screens, tabs and the drawer

4 The data model

5 The store: one place for all data

6 Flow: starting the app and saving data

7 Flow: making a sale

8 Flow: barcode scanning

9 Flow: utang (credit)

10 Flow: inventory and restocking

11 Flow: reports and the profit forecast

12 Flow: PIN lock, staff and voids

13 Security: encryption and backups

14 Look and language: theme and translations

15 Building and shipping

16 Important rules (the "arrangements")

17 Recipes: common changes, step by step

18 Glossary

> **How to read this**
>
> Each section names the real files, so keep VS Code open next to this guide. File paths are written like `src/store/StoreProvider.tsx`. Short code excerpts come straight from the app.

1 The big picture

Bantay is a point of sale, inventory and utang tracker for sari-sari stores. It is a single mobile app with **no server and no account**: everything lives on the phone.

What it does

- **Sell** by piece (tingi) or by pack, by tapping or by scanning barcodes; take cash, GCash or utang.
- **Track utang** per customer, with credit limits, payments and reminders.
- **Manage stock**: low-stock alerts, days-of-stock-left, restock and a shopping list.
- **Report**: totals, profit, best sellers, sales history, a forecast and a profit calculator.
- **Protect**: PIN lock, staff accounts with limits, an activity log, encrypted data and backups.

How the code is layered

Think of the app as five layers. Each layer only talks to the one below it.

Screens — src/app/... : what you see (Dashboard, Benta, Utang...)

▼ **useStore() / useT() / useTheme()**

Store — src/store/StoreProvider.tsx : all data + every action (checkout, restock...)

▼ **pure helper functions**

Selectors & libs — src/store/selectors.ts, src/lib/* : math, receipts, backups, crypto

▼ **encrypt → save**

Storage — src/store/storage.ts : SQLite key-value store on the phone

▼

Phone — Expo modules: SQLite, SecureStore, Camera, Print, Sharing, Crypto

Tech stack

Piece	What it's for
React Native + Expo SDK 57	One codebase for Android (and iOS/web). Expo provides the phone features.
Expo Router	Navigation from files: every file in `src/app/` is a screen.
TypeScript	Types catch mistakes before running (`npx tsc --noEmit`).
React Context	Shares the store with every screen (no Redux needed).
expo-sqlite (kv-store)	Saves the data on the phone.

expo-crypto + expo-secure-store	AES-256-GCM encryption; the key is kept in the Android Keystore / iOS Keychain.
expo-camera	Barcode scanner.
expo-print + expo-sharing	Receipts and reports as PDF; sharing backups.
expo-file-system + expo-document-picker	Writing and opening backup files.
react-native-reanimated	Animations (fade-ins, count-ups, spring buttons).
EAS Build	Builds the installable APK in the cloud.

Why offline-first?

Many sari-sari stores have weak or no internet. With no server, the app works anywhere, costs nothing to run, and the owner's customer data never leaves the phone (good for the Data Privacy Act). The trade-off: the owner must make backups.

2 Folder structure

```
Bantay-app/
├── app.json                ← app name, icon, package id, permissions, plugins
├── eas.json                ← build profiles (development / preview APK / production)
├── package.json            ← libraries and versions
├── assets/images/          ← app icon, splash screen, favicon
├── docs/                   ← privacy policy, terms, store checklist, guides
├── promo/                  ← separate Remotion project: promo video + images
└── src/
    ├── app/                ← SCREENS ONLY (Expo Router: file = route)
    │   ├── _layout.tsx     ← root: StoreProvider + Drawer + LockScreen
    │   ├── (tabs)/         ← bottom tabs group
    │   │   ├── _layout.tsx ← the 5 tabs
    │   │   ├── index.tsx   ← Dashboard
    │   │   ├── Benta.tsx   ← selling / cart / checkout
    │   │   ├── Inventory.tsx
    │   │   ├── Utang.tsx
    │   │   └── Reports.tsx
    │   ├── transactions.tsx ← every sale, by day, with receipts
    │   ├── history.tsx     ← sales by month and day
    │   ├── activity.tsx    ← who did what
    │   ├── settings.tsx    ← profile, look, backup, PIN, staff, data
    │   └── legal.tsx       ← privacy / terms / notices
    ├── components/         ← reusable UI pieces (BottomSheet, LockScreen…)
    ├── constants/theme.ts  ← colors, accents, radius, peso()
    ├── data/sample.ts      ← TYPES + sample data
    ├── i18n/               ← translations: index.ts, en.ts, wa.ts
    ├── legal/content.ts    ← legal text
    ├── lib/                ← non-UI helpers: backup, crypto, receipt…
    └── store/              ← StoreProvider, selectors, storage, theme hook
```

What lives where

Folder	Rule
`src/app/`	Only screens and `_layout.tsx` files. Anything else here would become a route by mistake.
`src/components/`	UI used by more than one screen, or big enough to deserve its own file.
`src/store/`	Data and logic. No styles here (except the theme hook).
`src/lib/`	Plain functions that talk to the phone or the outside world (files, PDF, crypto, internet).
`src/data/sample.ts`	The types for everything, plus the starter data.

Files with a `.web.ts` twin

`storage.ts` / `storage.web.ts` and `deviceKey.ts` / `deviceKey.web.ts`: Metro picks the `.web` version when running in a browser (uses `localStorage`) and the normal one on the phone (SQLite, SecureStore). The rest of the app imports `./storage` and never knows the difference.

The `@/` in imports (`import { useStore } from "@/store/StoreProvider"`) is a shortcut for `src/`, set in `tsconfig.json`.

3 Navigation: screens, tabs and the drawer

Expo Router turns files into screens. The nesting of `_layout.tsx` files decides how they are shown.

1. **Root layout**: `src/app/_layout.tsx`
   Wraps everything in `<StoreProvider>`, sets up the **Drawer** (☰ side menu), the header with the brand, and draws the **LockScreen** on top when the app is locked.

2. **Tabs layout**: `src/app/(tabs)/_layout.tsx`
   The bottom bar: Dashboard, Benta, Inventory, Utang, Reports. The `(tabs)` folder name in brackets is a *group*: it doesn't appear in the URL.

3. **Screens**: each file. Drawer-only screens (transactions, history, activity, settings, legal) sit next to `(tabs)`.

Hiding screens from staff

The drawer items are a list with an optional permission:

```
const DRAWER_TABS = [
  { label: 'Inventory', route: '/(tabs)/Inventory', icon: 'cube-outline', needs: 'inventory' },
  { label: 'Reports',   route: '/(tabs)/Reports',   icon: 'bar-chart-outline', needs:
'viewProfit' },
  ...
];
// only items the current user can() see are drawn
DRAWER_TABS.filter((tab) => !tab.needs || can(tab.needs))
```

Tabs use the same idea: `href: can('viewProfit') ? undefined : null`. A `null` href hides the tab. Screens that staff could still reach by a link also check at the top and show `<OwnerOnly />` instead (defence in depth).

Moving between screens

```
import { useRouter } from "expo-router";
const router = useRouter();
router.navigate("/(tabs)/Inventory");                          // go to a screen
router.navigate({ pathname: "/legal", params: { doc: "terms" } }); // with data

// on the other screen:
const { doc } = useLocalSearchParams<{ doc?: string }>();
```

> **Example: scanner → Inventory hand-off**
> When Benta scans an unknown barcode, it navigates to Inventory with `{ barcode, at }`. `at` is a timestamp so that scanning the same code twice still counts as a new request. Inventory waits until data is loaded (`ready`), then opens the "New product" form with the barcode filled in.

Route list

Route	File	Who can open
`/`	(tabs)/index.tsx (Dashboard)	Everyone
`/Benta`	(tabs)/Benta.tsx	Everyone
`/Inventory`	(tabs)/Inventory.tsx	Owner (permission `inventory`)
`/Utang`	(tabs)/Utang.tsx	Everyone
`/Reports`	(tabs)/Reports.tsx	Owner (`viewProfit`)
`/transactions`	transactions.tsx	Everyone
`/history`	history.tsx	Owner (`viewProfit`)
`/activity`	activity.tsx	Owner (`activity`)
`/settings`	settings.tsx	Owner (`settings`)
`/legal`	legal.tsx	Everyone (opened from Settings / setup)

4 The data model

All types live in `src/data/sample.ts`. Everything the app knows fits in one object called **State**:

```
type State = {
  products: Product[];      // what the store sells
  debtors:  Debtor[];       // customers with utang, each with a ledger
  sales:    Sale[];         // every sale ever (including voided ones)
  settings: Settings;       // store profile, language, theme, owner PIN
  closings: DayClosing[];   // end-of-day cash counts
  staff:    StaffMember[];  // name + PIN
  activity: ActivityEntry[];// log, newest first, max 1000
  meta: { lastBackupAt?, acceptedTermsAt?, ... };
};
```

Product

`id, name, price, cost, stock, min, unit, pack?, barcode?, category`

price and cost are **per piece**. `pack` = {name, size, price}, e.g. 1 tali = 10 sachets for ₱95.

Sale

`id, receiptNo, date, items[], total, payment, customerId?, gcashRef?, cashReceived?, by, voided?`

Each item is a **SaleItem**: a copy of the name, price, cost and pieces at the time of sale.

Debtor + LedgerEntry

`Debtor: id, name, phone?, limit, entries[]`

`LedgerEntry: type 'utang' | 'bayad' | 'void', amount, method?, note?`

Settings

`storeName, ownerName, ownerPhone, storeAddress, gcashNumber, defaultLimit, language, themeMode, accent, ownerPin`

Three design decisions to remember

Decision	Why
Stock is always counted in pieces. A pack is just "N pieces at a pack price".	Selling 1 tali of Kopiko takes 10 sachets off stock. One number, no conversions to get wrong.
Sales keep a copy of each item (name, price, cost) instead of pointing to the product.	If you rename a product, change its price or delete it, old receipts and reports don't change. They show what really happened.
Utang balance is computed, not stored. `balance = Σ utang − Σ bayad − Σ void`	The ledger is the truth. A stored balance could drift out of sync; a computed one can't.

The helper that turns a product into a sale line:

```
export function saleItem(p: Product, unit: SellUnit, qty: number): SaleItem {
  const pieces = unit === 'pack' && p.pack ? p.pack.size : 1;
  return {
    productId: p.id, name: p.name, unit,
    unitName: unit === 'pack' && p.pack ? p.pack.name : p.unit,
    pieces,
    price: unit === 'pack' && p.pack ? p.pack.price : p.price,
    cost: p.cost * pieces,
    qty,
  };
}
```

Voided sales are never deleted

They get a `voided` field and stay in the list (for proof). Every report skips them, e.g. `if (s.voided) continue;` in `summarize()`.

5 The store: one place for all data

`src/store/StoreProvider.tsx` is the heart of the app. It holds the State, loads and saves it, and offers every action screens need. Screens never change data directly; they call an action.

How a screen uses it

```
import { useStore } from "@/store/StoreProvider";

export default function Benta() {
  const { products, checkout, can } = useStore();  // read data + get actions
  ...
  const sale = checkout({ cart, payment: "cash", cashReceived: 200 });
}
```

The actions

Action	What it does
`checkout(input)`	Records a sale, takes stock off, adds utang if needed, logs it, returns the Sale (for the receipt).
`voidSale(id, reason)`	Marks a sale voided, puts stock back, reverses its utang.
`restock(id, qty)`	Adds pieces to a product.
`addProduct` / `updateProduct` / `removeProduct`	Manage products. Updates log only the fields that changed.
`addDebtor` / `addLedgerEntry` / `updateDebtorLimit`	Manage utang customers and their entries.
`updateSettings(patch)`	Change settings; turning on a PIN keeps the owner logged in.
`addStaff` / `removeStaff`	Staff accounts.
`login(pin)` / `lock()` / `can(permission)`	Session and permissions (section 12).
`closeDay(expected, counted)`	Saves an end-of-day cash count.
`markBackedUp` / `restoreData` / `resetData` / `startFresh` / `wipeData`	Backup bookkeeping and data reset options.

The golden rule: immutable updates

React only redraws when it sees a **new** object. So every action builds new arrays instead of editing old ones:

```
restock(productId, qty) {
  setState((s) => logged(
    { ...s, products: s.products.map((p) =>
        p.id === productId ? { ...p, stock: p.stock + qty } : p) },
    'restock',
    `${productName(productId)} +${qty}`,
  ));
}
```

- `{ ...s, products: … }` copies the state and replaces only `products`.
- `.map` makes a new array; the changed product becomes a new object.
- `logged(state, kind, text)` adds an activity entry (who, when, what) and keeps the log at 1000 entries.

Derived values, computed on the fly

The store also exposes `ready` (data loaded), `storageError` (saved data couldn't be read), `session` (who is logged in) and `locked`. Totals, balances and forecasts are **not** stored; screens compute them with the selectors in `src/store/selectors.ts`, for example `summarize(todaySales)` or `balanceOf(debtor)`.

Key selectors

Function	Returns
`summarize(sales)`	total, profit, transactions, totals by payment, best sellers (skips voided)
`trend(sales, period)`	bars for the Today / Week / Month charts
`balanceOf(debtor)`, `totalUtang(debtors)`	utang owed
`isLowStock(p)`	`p.stock <= p.min`
`dailyRate`, `daysLeft`	pieces sold per day (last 7 days) and days until it runs out
`restockList(products)`	the shopping list, rounded up to whole packs
`smartTip`	the tip on the Dashboard ("Nescafe runs out today...")
`monthlyHistory`, `dailyHistory`	Sales history screen
`forecast(sales)`	next 7/30 days, this month, trend, best weekday (section 11)

6 Flow: starting the app and saving data

When the app opens

1. **Start with sample data in memory** and `ready = false`. The lock screen covers the app while loading.
   StoreProvider: useState(seedState)
2. **Load the saved text** from the phone (key `bantay-state-v2`).
   loadItem() in storage.ts
3. **Decrypt** if it starts with `enc1:`, using the device key from SecureStore. (Old versions saved plain JSON; that still loads.)
   decryptText() in lib/crypto.ts, getDeviceKey() in store/deviceKey.ts
4. **normalizeState()** fills in fields added in newer versions, so old data and old backups still work. It also applies one-time changes (e.g. light theme and English became the defaults).
5. **ready = true.** If an owner PIN is set, the LockScreen stays until someone logs in. If no data was saved yet, the Dashboard shows the welcome/setup sheet (language, store name, terms, sample vs empty).

> **If decrypting fails**
> `storageError` becomes true, a red card appears on the Dashboard, and **saving is paused** so the unreadable data isn't overwritten. The owner can restore a backup from Settings.

Every time data changes

1. An action calls `setState` → React redraws the screens with the new data.
2. A `useEffect` watching `state` turns it into JSON, encrypts it, and saves it with the `enc1:` prefix.
3. Saves go through a **queue** (`saveQueue`), one after another, so an older, slower save can never finish last and overwrite newer data.

```
useEffect(() => {
  if (!ready || storageError) return;          // never save before loading finished
  const json = JSON.stringify(state);
  saveQueue.current = saveQueue.current
    .then(async () => {
      const encrypted = await encryptText(json, await getDeviceKey());
      await saveItem(STORAGE_KEY, ENCRYPTED_PREFIX + encrypted);
    })
    .catch(() => {});
}, [state, ready, storageError]);
```

Why "if (!ready) return"?

Without it, the first save would write the sample data over the real saved data before loading finished. Always load first, then save.

7 Flow: making a sale

Files: `src/app/(tabs)/Benta.tsx`, `checkout` in `StoreProvider.tsx`, `src/components/ReceiptSheet.tsx`.

1. **Build the cart.** Tapping a product adds 1 piece; the pack button adds 1 pack. The cart is an object keyed by product and unit: `{ "p1:tingi": 2, "p3:pack": 1 }`. The screen checks stock (in pieces) before adding.
2. **Open the cart sheet.** Change quantities, remove a line (✕) or everything ("Remove all"). Pick **Cash** (with cash received → change), **GCash** (shows your GCash number, optional reference) or **Utang** (pick or add a customer; warns if over the credit limit).
3. **Validate.** Not enough cash? No customer for utang? An alert explains and nothing is saved. A short lock (`finishing` ref) stops a double-tap from recording the sale twice.
4. **checkout()** in the store:
   1. Turns each cart line into a SaleItem with `saleItem()`.
   2. Adds up the total.
   3. For utang: creates the customer if new, adds an `utang` ledger entry.
   4. Gives the next receipt number (highest so far + 1).
   5. Takes the pieces off stock (`adjustStock(products, sale, -1)`).
   6. Saves the sale with who made it (`by`) and logs "#001561 · ₱163.00".
5. **Receipt.** The ReceiptSheet shows the sale; Print / Share make a 58 mm-wide PDF (`lib/receipt.ts`). Every receipt says "Hindi ito opisyal na resibo (BIR)".
6. **Everything updates by itself**: Dashboard totals, stock, low-stock alerts, the customer's balance, Transactions, Reports. They all read the same State.

8 Flow: barcode scanning

Files: `src/components/BarcodeScanner.tsx`, `onScanned` in Benta, `handleNewBarcode` in Inventory, `src/lib/productLookup.ts`.

1. **Open the scanner.** It asks for camera permission once, then shows `CameraView` from expo-camera with a scan box. Supported: EAN-13/8, UPC-A/E, Code 128/39, QR.
2. **A code is read** → `onBarcodeScanned`. The same code is ignored for 1.5 s (`REPEAT_COOLDOWN_MS`) so one item isn't added many times per second.
3. **In Benta (continuous mode):** known barcode → +1 piece in the cart and a green "Coke 290ml ×1 · ₱20.00" bar. Unknown → "Unknown: 480..." and, for the owner, an **Add to Inventory** button.
4. **In Inventory:** known → opens that product. Unknown → opens "New product" with the barcode filled in, then asks **Open Food Facts** (free online database) for the name and category. Offline? It's skipped and the owner types the name.

> **Only the barcode number goes online**
> The lookup sends just the digits to openfoodfacts.org. No store or customer data ever leaves the phone. This is stated in the Privacy Policy.

> **Known behaviour**
> Holding the same item in front of the camera for longer than 1.5 s adds it again. That's intended for scanning several of the same item, but cashiers should move each item away after the beep.

9 Flow: utang (credit)

Files: `src/app/(tabs)/Utang.tsx`, `addDebtor` / `addLedgerEntry` in the store, `balanceOf` in selectors.

1. **Utang is created** two ways: a sale paid with "Utang" (automatic entry, note "3 items"), or "Add customer" with an opening amount.
2. **Balance** = utang − bayad − void, from the ledger. The list shows each customer's balance, last payment and last utang; the dark card shows the store's total.
3. **Payments** ("Bayad", cash or GCash) add a `bayad` entry. You can't pay more than the balance.
4. **Limits and reminders.** Each customer has a credit limit (default from Settings); checkout warns before going over it. "Remind" opens the share menu with a friendly message (and your GCash number). "Print" makes a PDF of the whole utang list.

10 Flow: inventory and restocking

File: `src/app/(tabs)/Inventory.tsx` (owner only).

- **Summary card:** "Cost of stock" = Σ cost × stock; "If all sells" = Σ price × stock, and the profit in between.
- **Each row:** a stock bar coloured green / orange (low, `stock ≤ min`) / red (out).
- **Tap a product** → a sheet with stock, minimum, profit per piece, a "days left" estimate (`daysLeft`), the barcode (tap to link one), quick restock buttons (+1/+2/+5 packs, or +6/+12/+24) and **Edit product**.
- **Add / edit form,** step by step: piece name → selling price → cost (**per piece** or **per box**: "6 cans for ₱500" → ₱83.33 each, stored with 4 decimals so 6 × cost adds back to ₱500) → optional pack selling → stock and reminder level. A live "Profit: ₱... per piece" box helps catch mistakes. Barcodes must be unique.
- **Delete product:** removes it from the list; old sales keep their copy, so nothing breaks.
- **Shopping list:** `restockList()` suggests enough to reach twice the minimum, rounded up to whole packs, and shares it as text.

11 Flow: reports and the profit forecast

Files: `src/app/(tabs)/Reports.tsx`, `src/app/history.tsx`, `src/components/ForecastCard.tsx`, `forecast()` in selectors.

- **Reports:** Today / Week / Month totals, profit and margin, average per sale, a trend chart, best sellers, payment breakdown, "Share report" and **End of day**: expected cash (cash sales + cash utang payments + starting cash) vs counted cash, plus expected GCash.
- **Sales history:** the last 6 months as bars; tap a month to see each day, compared with the same days of the previous month (fair comparison mid-month).

How the forecast works

1. Take the last **28 days** of sales (not voided, not today).
2. Average sales **per weekday**: "a typical Sunday sells ₱1,980". Only weekdays the store has history for count (a new store isn't punished for empty days before it started).
3. **Next 7 / 30 days** = add up the weekday averages for the days ahead. **This month** = sold so far + the rest of today + the remaining days.
4. **Profit** = forecast × the last 28 days' profit margin. **Trend** = last 14 days vs the 14 before. **Best day** = the weekday with the highest average.

The **profit calculator** answers "if I sell N of this product (by piece or pack), how much do I earn?" using `saleItem()` for the numbers.

> **Simple on purpose**
> A weekday average is easy to explain to a store owner and works with little data. The Terms of Use say forecasts are estimates, not promises.

12 Flow: PIN lock, staff and voids

Files: `StoreProvider.tsx` (session, `can`, `login`), `src/components/LockScreen.tsx`, `OwnerOnly.tsx`, `ReceiptSheet.tsx`.

Sessions

- No owner PIN → there is only the owner; no lock screen.
- Owner PIN set → `locked` is true until someone enters a PIN. The **session** remembers only `{ id, role }`; the name is looked up live, so a rename shows at once and a **removed staff member is logged out**.
- The session is **not saved**: closing the app asks for the PIN again. After **5 minutes in the background** the app locks itself (`AUTO_LOCK_MS`, using `AppState`).

Permissions

```
type Permission = 'viewProfit' | 'inventory' | 'manageProducts'
                | 'voidSale' | 'settings' | 'backup' | 'activity';

const STAFF_BLOCKED: Permission[] = [ ...all of the above ];

can(permission) {
  return current.role === 'owner' || !STAFF_BLOCKED.includes(permission);
}
```

Staff can sell, see transactions and record utang, but can't see profit, inventory, history, settings or the activity log, can't manage products, and can't void.

Voiding a sale

1. Open the sale's receipt (Transactions or right after selling) → **Void this sale** → choose or type a reason.
2. If staff is logged in, the **owner's PIN** is required.
3. `voidSale()`: puts every piece back on the shelf (`adjustStock(…, +1)`), adds a `void` ledger entry if it was utang, marks the sale `voided: { date, by, reason }`, and logs it.

Activity log

Every action passes through `logged()`: sales, voids, restocks, product edits (only the changed fields), utang, payments, logins, settings (never PIN values), backups, restores. The Activity screen groups it by day with filters.

13 Security: encryption and backups

Data on the phone

What	How	File
The key	A random AES-256 key made on first save, kept in **SecureStore** (Android Keystore / iOS Keychain), separate from the data.	`store/deviceKey.ts`
The data	JSON → UTF-8 bytes → **AES-256-GCM** (random IV + auth tag) → base64 → saved with the `enc1:` prefix.	`lib/crypto.ts`
Integrity	GCM refuses to decrypt changed data, so tampering is detected, not silently loaded.	

> **Bug we fixed (worth remembering)**
> On Android, `AESSealedData.fromCombined()` only accepts **bytes**. Passing the base64 string worked on web and iOS but failed on Android, so the app showed "Hindi mabasa ang naka-save na data". The fix converts first: `fromCombined(base64ToBytes(text))`. Lesson: test native code on a real Android phone.

Backup files

```
// plain backup
{ "app": "bantay", "version": 2, "exportedAt": "…", "data": { …State… } }

// password-protected backup
{ "app": "bantay", "version": 2, "exportedAt": "…", "encrypted": true,
  "salt": "…", "iterations": 20000, "data": "<AES-GCM ciphertext>" }
```

- **Make:** Settings → Back up now → optional password → the share menu (Bluetooth, Messenger, Drive, SD card).
- **Password key:** 20,000 rounds of salted SHA-256 (expo-crypto has no PBKDF2). A forgotten password can't be recovered.
- **Restore:** pick the file → check it's a Bantay backup with products/sales/debtors arrays → ask for the password if locked → confirm → `restoreData()` runs `normalizeState()`, so older backups load too.
- The Dashboard reminds the owner if there's no backup for 7 days.

Other privacy points

- No accounts, ads, analytics or tracking. The only internet use is the barcode lookup.
- Camera permission only; microphone is blocked in `app.json`.
- Settings → Data: "Start with an empty store" (keeps the profile) and "Delete all data" (asks twice).

14 Look and language: theme and translations

Theme

`src/constants/theme.ts` defines `lightColors` and `darkColors` (background, surface, text, muted, border, red, orange, blue…) and 8 **accents** (green, blue, violet…) for `primary`. `useTheme()` picks light/dark (Settings: Light, Dark or follow the phone; light is the default) and applies the accent.

Every screen builds its styles from the colors, once per color change:

```
const { colors } = useTheme();
const styles = useMemo(() => createStyles(colors), [colors]);
...
const createStyles = (colors: Colors) => StyleSheet.create({
  screen: { flex: 1, backgroundColor: colors.background },
  title:  { fontSize: 26, fontWeight: "700", color: colors.text },
});
```

> **Rule**
> Never write a color like `"#111827"` in a screen when there's a theme color for it; otherwise dark mode breaks. (Money and "OK" signals use the fixed `success` green on purpose.)

Translations

The Filipino text **is** the key. `en.ts` maps it to English, `wa.ts` to Waray (missing Waray falls back to Filipino).

```
// src/i18n/en.ts
'Benta ngayon': 'Sales today',
'Kulang pa ng {amount}.': 'Short by {amount}.',

// in a screen
const t = useT();
t("Benta ngayon")                          // "Sales today" / "Benta ngayon" / "Baligya yana nga
adlaw"
t("Kulang pa ng {amount}.", { amount: peso(12) })
```

`TKey = keyof typeof en`, so a typo or a missing English line is a **TypeScript error**. To add new text: write the Filipino in `t("…")`, add the English to `en.ts`, optionally the Waray to `wa.ts`.

Small reusable pieces

Component	Used for
`BottomSheet`	Every sheet (cart, forms, receipts); slide or fade.
`ReceiptSheet`	Receipt view, print/share, void flow.
`BarcodeScanner`	Camera scanner, single or continuous.

`LockScreen`	PIN keypad with shake on a wrong PIN.
`OwnerOnly`	"Owner only" placeholder for blocked screens.
`ForecastCard`	Forecast + profit calculator.
`BrandTitle`	Logo + "Bantay" + screen name in the header.
`motion.tsx`	`FadeIn`, `PressableScale`, `GrowBar`, `useCountUp`.
`lib/confirm.ts`	`confirm()` / `notify()` that work on phone and web.

15 Building and shipping

Task	Command
Run while coding (Expo Go)	`npx expo start`
Check types	`npx tsc --noEmit`
Add a library	`npx expo install <package>` (picks the version that matches SDK 57)
Fix library versions	`npx expo install --fix`
Installable APK	`npx eas-cli@latest build --platform android --profile preview`
Play Store bundle (.aab)	`npx eas-cli@latest build --platform android --profile production`

Config files

- **app.json:** name "Bantay", slug `bantay`, package `com.porkarie.bantay` (can't change after the first Play upload), icon and splash, camera permission text, blocked microphone, iOS privacy manifest, and the plugins (expo-router, splash-screen, sqlite, camera, sharing, secure-store).
- **eas.json:** `development`, `preview` (APK for sharing) and `production` (auto-incremented version code).
- There are no `android/` or `ios/` folders: Expo generates them during the build (Continuous Native Generation). Change native settings in `app.json`, never by hand.

> **Updating users**
> A new APK built from the same EAS project is signed with the same key, so it installs over the old one and keeps the data. Raise `"version"` in app.json so people can tell versions apart.

Store release steps (privacy URL, Data safety answers, BIR and GCash notes) are in `docs/store-release-checklist.md`.

16 Important rules (the "arrangements")

#	Rule	Because
1	Only screens and `_layout.tsx` go in `src/app/`.	Every file there becomes a route.
2	Screens change data only through store actions.	One place for logic, logging and saving.
3	Never mutate state; build new arrays/objects.	React only redraws on new objects.
4	Stock is in pieces; packs are pieces × size.	One unit, no conversion bugs.
5	Sales copy item details; never look up old prices from products.	History must not change when products change.
6	Don't delete sales; void them. Reports skip `voided`.	Proof and an audit trail.
7	Compute totals and balances; don't store them.	Stored copies drift out of sync.
8	Save only after `ready`; pause on `storageError`.	Never overwrite real data with sample or broken data.
9	New State fields need a default in `normalizeState()`.	Old saved data and backups must still load.
10	Check permissions twice: hide the link **and** guard the screen.	Links can be reached in other ways.
11	All visible text goes through `t()` with an English line in `en.ts`.	Three languages; TypeScript catches gaps.
12	Use theme colors and `createStyles(colors)`.	Dark mode and accents keep working.
13	Add libraries with `npx expo install`.	Versions must match Expo SDK 57.
14	Read the Expo docs for **your** SDK version.	Expo APIs change between versions.
15	Run `npx tsc --noEmit` and test on Android before building.	Some bugs only appear on a real phone (see section 13).

17 Recipes: common changes, step by step

Add a new screen (e.g. "Suppliers")

1. Create `src/app/suppliers.tsx` with a default-exported component.
2. Add `<Drawer.Screen name="suppliers" />` in `src/app/_layout.tsx`.
3. Add an item to `DRAWER_TABS` (label, route, icon, optional `needs`).
4. Add the label's English to `en.ts`. Run `npx tsc --noEmit`.

Add a field to products (e.g. "expiry date")

1. Add `expiry?: string` to `Product` in `src/data/sample.ts` (optional, so old data still fits).
2. Add the input to the product form in `Inventory.tsx` (`EMPTY_FORM`, `openEdit`, `saveProduct`).
3. Show it where needed; add a selector like `isExpiringSoon(p)` in `selectors.ts`.

Add a new action (e.g. "change a price for everything in a category")

1. Add its type to the `Store` type in `StoreProvider.tsx`.
2. Write it inside `store` with `setState((s) => logged({ ...s, products: s.products.map(...) }, 'product', '...'))`.
3. Call it from a screen via `useStore()`.

Add something only the owner may do

1. Add a name to the `Permission` type and to `STAFF_BLOCKED`.
2. Use `can("yourPermission")` to hide the button, and guard the screen with `<OwnerOnly />`.

Change the app's colors

1. Edit `lightColors` / `darkColors` or add an accent to `ACCENTS` in `src/constants/theme.ts`.
2. New accents appear automatically as swatches in Settings.

Translate a new language

1. Add the code to `Language` in `sample.ts` (e.g. `'ceb'`).
2. Create `src/i18n/ceb.ts` like `wa.ts`, handle it in `translate()`, add it to `LANGUAGES`.
3. Add month and weekday names in `selectors.ts` (`MONTHS`, `WEEKDAYS`, `MONTH_NAMES`); TypeScript will point to each one.

18 Glossary

Word	Meaning in this app
Tingi	Selling by the piece (one sachet, one stick).
Pack / tali / kahon	A bigger selling unit made of N pieces.
Utang / bayad	Credit given to a customer / a payment toward it.
Suki	A regular customer.
Puhunan / tubo	Cost / profit.
State	The single object holding all the store's data.
Store / StoreProvider	The component that holds State and its actions, shared through Context.
Action	A store function that changes data (checkout, restock…).
Selector	A pure function that computes something from data (summarize, balanceOf…).
Session	Who is logged in right now (owner or a staff member).
Permission	Something staff may or may not do (viewProfit, voidSale…).
Void	Cancelling a sale while keeping its record.
Ledger	The list of utang/bayad/void entries for one customer.
normalizeState	Fills in missing fields so older data still loads.
AES-256-GCM	The encryption used for saved data and locked backups.
EAS	Expo Application Services: builds the APK in the cloud.
Expo Go	The testing app that runs your code while `npx expo start` is on.

Next step

To practise, follow `docs/learn-bantay-step-by-step.md` (42 days). It rebuilds the app piece by piece and points back to the files explained here.